

AI-Driven Multidisciplinary Design Optimization

AIAA MDO TC Workshops on Multifidelity Methods

Dr. Ahmed Bayoumy

Siemens Digital Industries Software

June 5, 2026



Contents

- 1 Introduction
- 2 AI/ML in MDO
- 3 Case Studies
- 4 Tools & Frameworks
- 5 Exercises
- 6 Future Directions



Outline

1 Introduction

2 AI/ML in MDO

3 Case Studies

4 Tools & Frameworks

5 Exercises

6 Future Directions



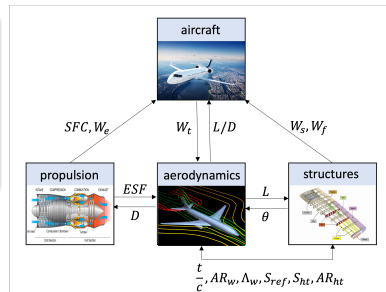
What is Multidisciplinary Design Optimization?

Definition

MDO is a methodology for designing complex engineering systems that are governed by mutually interacting **multiple disciplines** simultaneously.

Key characteristics:

- Coupled physics / simulation models
- Large design spaces
- Conflicting objectives & constraints
- High computational cost

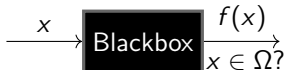


Why is MDO Hard? Blackbox Optimization Challenges

We consider blackbox optimization problems:

$$\min_{x \in \Omega} f(x)$$

where evaluations of f and the functions defining Ω are usually the result of a computer code (a blackbox).



Each call to the simulation is time-consuming



Why is MDO Hard? Blackbox Optimization Challenges

We consider blackbox optimization problems:

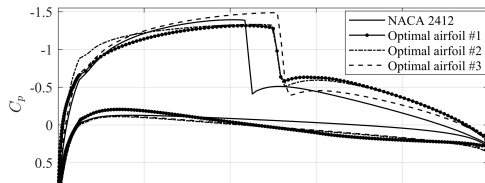
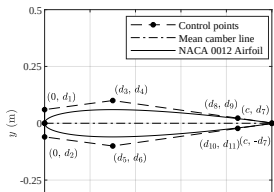
$$\min_{x \in \Omega} f(x)$$

where evaluations of f and the functions defining Ω are usually the result of a computer code (a blackbox).



Each call to the simulation is time-consuming

Transonic airfoil shape optimization using TSFOIL, SU2, OpenFoam, fluent ...etc.



Why is MDO Hard? Blackbox Optimization Challenges

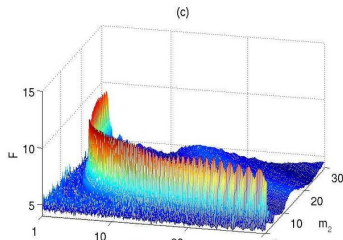
We consider blackbox optimization problems:

$$\min_{x \in \Omega} f(x)$$

where evaluations of f and the functions defining Ω are usually the result of a computer code (a blackbox).



Derivatives cannot be trusted or evaluated.



Why is MDO Hard? Blackbox Optimization Challenges

We consider blackbox optimization problems:

$$\min_{x \in \Omega} f(x)$$

where evaluations of f and the functions defining Ω are usually the result of a computer code (a blackbox).



BBO challenges



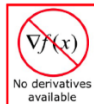
Long runtime



Large memory requirement



Software might fail



No derivatives available



Local optima



Non-smooth, noisy

Possible bad properties of a blackbox from *J. Simonis, 2009, The Boeing Company*



Why is MDO Hard? Blackbox Optimization Challenges

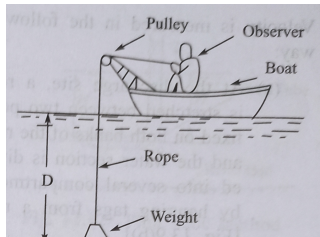
We consider blackbox optimization problems:

$$\min_{x \in \Omega} f(x)$$

where evaluations of f and the functions defining Ω are usually the result of a computer code (a blackbox).



John Dennis analogy [Powell 1994]



Why Is MDO Hard? Multidisciplinary challenges

Computational challenges

- Each discipline requires a costly solver
- Design space grows exponentially with variables
- Gradient computation across couplings is non-trivial

Organizational challenges

- Siloed engineering teams
- Inconsistent data formats
- Trade-off transparency

The AI Opportunity

Machine learning can dramatically reduce evaluation cost, enable global search, and learn cross-disciplinary couplings from data.



AI use in Classical MDO Architectures

Architecture	Coupling strategy	Typical use	AI use
MDF	Fixed-point iteration	Tightly coupled systems	Surrogate modeling
IDF	Consistency constraints	Parallel disciplines	Bayesian optimization
AAO	All-at-once	Gradient-rich problems	Differentiable surrogates
CSSO	Coordination	Large organizations	RNNs and GRUs
BLISS	Sensitivity-based	Hierarchical systems	Sensitivity learning

Table: Common MDO architectures, their trade-offs and AI use as surrogate models.



Outline

- 1 Introduction
- 2 AI/ML in MDO
- 3 Case Studies
- 4 Tools & Frameworks
- 5 Exercises
- 6 Future Directions



The AI Toolkit for MDO

Machine learning models

- Neural Networks (BNN, FFNN, RNN)
- Kernel methods (GP, RBF)
- Tree-based models (Random Forest, XGBoost)
- Ensemble methods (stacking, boosting)

Derivative-Free Optimization

- Direct search (MADS, GPS)
- Evolutionary algorithms (GA)
- Heuristic methods (SA, PSO)
- Bayesian Optimization

Gradient-Based (Differentiable)

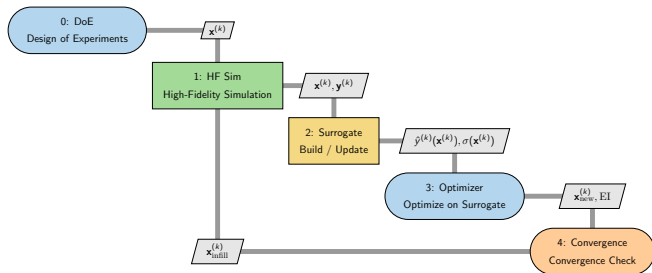
- Adjoint methods
- Physics-Informed Neural Nets
- Differentiable Simulators

Reinforcement Learning

- Bootstrapping
- Uncertainty-aware exploration
- Multi-agent discipline coordination



Surrogate-Based Optimization (SBO) Workflow



Key steps: Sample $\rightarrow$ Simulate $\rightarrow$ Surrogate $\rightarrow$ Optimize $\rightarrow$ Check convergence

- SBO is a powerful framework for expensive MDO problems.
- Surrogate quality and infill strategy are critical.
- Key metric: **Expected Improvement (EI)** or **Probability of Feasibility**
- Infill criterion balances *exploitation* and *exploration*



Gaussian Process Surrogate — Quick Recap

A GP defines a distribution over functions:

$$f(\mathbf{x}) \sim \mathcal{GP}(\mu(\mathbf{x}), k(\mathbf{x}, \mathbf{x}'))$$

Prediction

$$\begin{aligned}\mu^* &= \mathbf{k}_*^\top \mathbf{K}^{-1} \mathbf{y} \\ \sigma^{*2} &= k_{**} - \mathbf{k}_*^\top \mathbf{K}^{-1} \mathbf{k}_*\end{aligned}$$

Common kernels

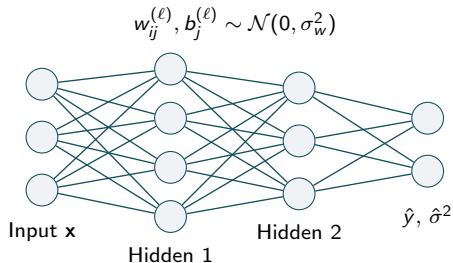
- Squared Exponential (RBF)
- Matérn 5/2
- Rational Quadratic

Key insight

σ^* gives a **free uncertainty estimate** — essential for active learning in MDO.



Bayesian Neural Network (BNN): Architecture and Probabilistic Prediction



$$p(y | \mathbf{x}, \mathcal{D}) = \int p(y | \mathbf{x}, \boldsymbol{\theta}) p(\boldsymbol{\theta} | \mathcal{D}) d\boldsymbol{\theta}$$

$$p(y | \mathbf{x}, \mathcal{D}) \approx \frac{1}{S} \sum_{s=1}^S p(y | \mathbf{x}, \boldsymbol{\theta}^{(s)}), \quad \boldsymbol{\theta}^{(s)} \sim p(\boldsymbol{\theta})$$

$$P(y \leq y_{\text{req}} | \mathbf{x}, \mathcal{D}) \approx \frac{1}{S} \sum_{s=1}^S \mathbb{I}(y^{(s)} \leq y_{\text{req}})$$

Quality/criteria probability

For pass/fail criteria (e.g., stress $\leq \sigma_{\text{max}}$), BNN outputs a full predictive distribution, enabling direct risk-aware decisions.



Physics-Informed Neural Networks (PINNs)

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{data}}}_{\text{observations}} + \lambda \underbrace{\mathcal{L}_{\text{PDE}}}_{\text{residual}} + \mu \underbrace{\mathcal{L}_{\text{BC}}}_{\text{boundaries}}$$

- Embed governing equations directly in the loss function
- Works with sparse / noisy data
- Suitable for **inverse problems** & sensitivity analysis
- Libraries: DeepXDE, Modulus (NVIDIA), JAX

MDO advantage

PINNs can serve as differentiable discipline models, enabling gradient propagation across the entire coupled system.



Agentic AI in MDO

What is an agentic workflow?

- LLM-based planner decomposes design goals into tasks
- Tool-using agents call solvers, scripts, and databases
- Memory stores prior runs, constraints, and failures
- Verifier agent checks feasibility before promotion

Closed-loop cycle

Plan → Simulate → Learn → Propose → Verify

- Human sets objectives, safety limits, and trust gates
- Agent proposes candidates and justifies trade-offs

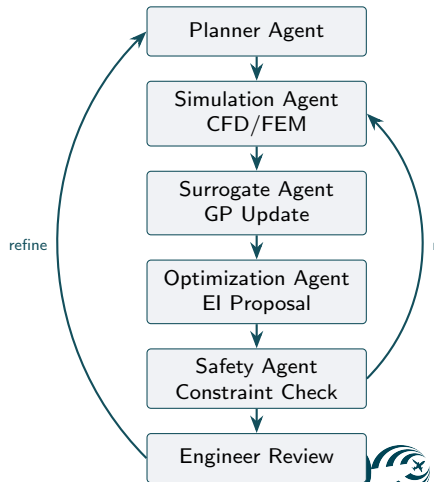
Why it matters for MDO

Agentic AI can orchestrate multi-disciplinary pipelines, reduce manual iteration overhead, and accelerate convergence while keeping engineers in the loop.

Agentic AI in MDO: Illustrative Example

Scenario: Wing drag minimization with lift and stress constraints

- ❶ **Planner agent** decomposes goals and constraints
- ❷ **Simulation agent** runs CFD/FEM on candidates
- ❸ **Surrogate agent** updates GP mean and uncertainty
- ❹ **Optimization agent** selects EI-driven infill points
- ❺ **Safety agent** filters infeasible proposals
- ❻ **Reviewer agent** sends ranked options to engineer



Related references: Yao et al., *ReAct* (ICLR 2023); Wu et al., *AutoGen* (2023); Martins & Ning, *Engineering Design*



Illustrative Example: Wing Design

$$\min_{\mathbf{x}} \quad f(\mathbf{x}) = C_d(\mathbf{x})$$

$$\text{s.t.} \quad g_1(\mathbf{x}) = |C_l(\mathbf{x}) - 1.2| - 0.1 \leq 0,$$

$$g_2(\mathbf{x}) = \sigma(\mathbf{x}) - 800 \leq 0,$$

$$\mathbf{x}^L \leq \mathbf{x} \leq \mathbf{x}^U$$

$$\mathbf{x} = [\text{AR}, \text{TR}, \text{tc}]^\top, \quad \mathbf{x}^L = [5.0, 0.2, 0.08]^\top, \quad \mathbf{x}^U = [15.0, 0.8, 0.15]^\top$$

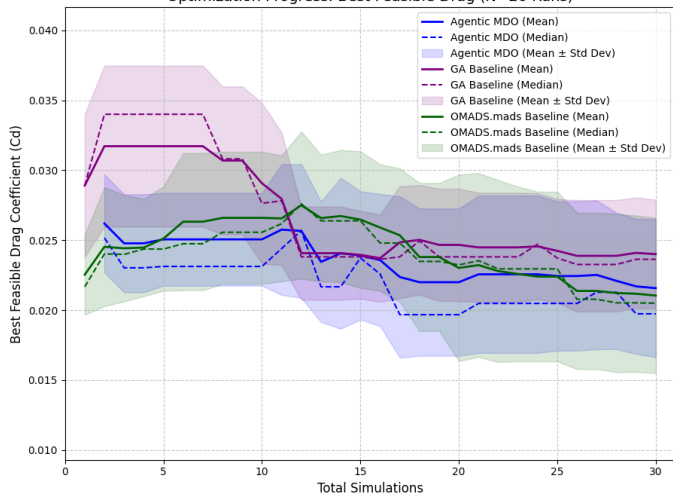
$$C_d(\mathbf{x}) = 0.02 + \frac{0.01}{\text{AR}} + 0.05 \text{tc}^2 + 0.01(1 - \text{TR})$$

$$C_l(\mathbf{x}) = 0.5 + 0.05 \text{AR} + 0.1 \text{TR} - 0.2 \text{tc}, \quad \sigma(\mathbf{x}) = 100 \text{tc}^{-1} + 5 \text{AR} + 20(1 - \text{TR})$$



Agentic MDO Workflow

Optimization Progress: Best Feasible Drag (N=20 Runs)



LLM Engagement: Orchestration Layer

Planner Agent

- Orchestrates phases: sampling, optimization, and review
- Issues goals to numerical agents (CFD/FEM/surrogate)
- Reports progress and explains next-step rationale

Reviewer Agent

- Interprets feasible candidate designs
- Ranks alternatives by objective value and risk
- Produces concise engineer-facing recommendations

Division of Labor

The LLM handles strategy, coordination, and communication; numerical solvers and optimizers handle heavy computation.



AI-assisted Constrained Optimization

Constrained Optimization Problem

$$\begin{aligned} \min_{\mathbf{x} \in [\mathbf{x}^L, \mathbf{x}^U]} \quad & f(\mathbf{x}) \\ \text{s.t.} \quad & g_j(\mathbf{x}) \leq 0, \quad j = 1, \dots, m \end{aligned}$$

Deep-Ensemble NNs (Heteroscedastic)

$$\mathcal{D}_t = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^{N_t}, \quad \mathbf{y} = [f, \text{metrics}]^\top$$

$$(\boldsymbol{\mu}^{(k)}(\mathbf{x}), \log \sigma^{2,(k)}(\mathbf{x})) = \text{NN}_k(\mathbf{x}), \quad k = 1, \dots, K$$

$$\mathcal{L}^{(k)} = \frac{1}{2} \sum_r \left(\log \sigma_r^{2,(k)} + \frac{(y_r - \mu_r^{(k)})^2}{\sigma_r^{2,(k)}} \right)$$

$$\bar{\boldsymbol{\mu}} = \frac{1}{K} \sum_{k=1}^K \boldsymbol{\mu}^{(k)}, \quad \bar{\sigma}^2 = \frac{1}{K} \sum_{k=1}^K \left(\sigma^{2,(k)} + (\boldsymbol{\mu}^{(k)})^2 \right) - \bar{\boldsymbol{\mu}}^2$$



Risk Estimates and Hybrid Evaluation Policy

$$\hat{p}_{\text{inf}}(\mathbf{x}) = \mathbb{P}(\exists j : g_j(\mathbf{x}) > 0), \quad \hat{p}_{\text{no-imp}}(\mathbf{x}) = \mathbb{P}(f(\mathbf{x}) \geq f^* - \delta)$$

$$f^* = \min \{f(\mathbf{x}_i) : g_j(\mathbf{x}_i) \leq 0, \forall j\}$$

- If $N_{\text{sim}} < N_0$: run true simulation
- Else if $s < s_{\text{max}}$ and model is trained: use prediction
- Otherwise: run simulation and retrain every R new simulations

Counters: N_{sim} (true evaluations), s (consecutive predicted evaluations).



Methodology applied to Wing Design Problems

Surrogate targets and uncertainty:

$$\mathbf{y}(\mathbf{x}) = [C_d, C_l, \sigma]^\top, \quad \hat{\mathbf{y}}^{(k)} \sim \mathcal{N}(\boldsymbol{\mu}^{(k)}(\mathbf{x}), \text{diag}(\boldsymbol{\sigma}^{2,(k)}(\mathbf{x})))$$

$$\bar{\boldsymbol{\mu}}, \bar{\boldsymbol{\sigma}}^2 \quad \text{from ensemble moment matching}$$

Risk metrics used in logging/decision support:

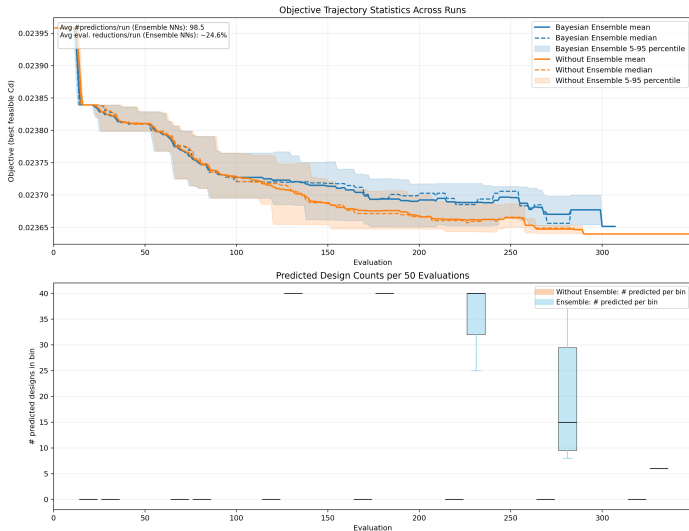
$$\hat{p}_{\text{inf}}(\mathbf{x}) = \mathbb{P}(|C_l - 1.2| > 0.1 \vee \sigma > 800)$$

$$\hat{p}_{\text{no-imp}}(\mathbf{x}) = \mathbb{P}(C_d \geq C_d^* - \delta), \quad C_d^* = \min_{\text{feasible sampled } i} C_{d,i}$$

Optimization loop (OMADS + surrogate): Initial true evaluations $\rightarrow$ train surrogate $\rightarrow$ alternate predicted/true evaluations with capped predicted streak and periodic retraining.



Results: Wing Design Validation



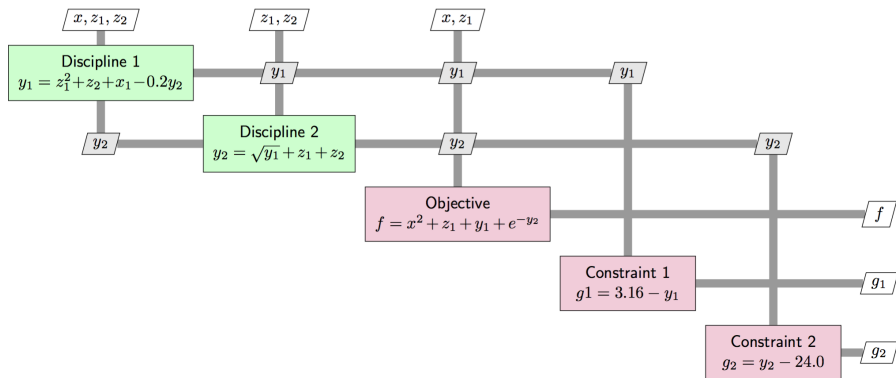
Outline

- 1 Introduction
- 2 AI/ML in MDO
- 3 Case Studies**
- 4 Tools & Frameworks
- 5 Exercises
- 6 Future Directions



Case Study 1 — Sellar Benchmark Problem

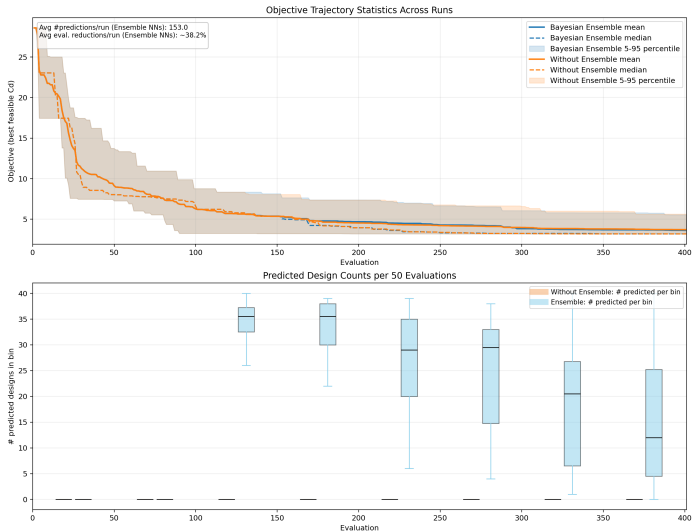
Purpose: compact benchmark for coupled-discipline MDO.



XDSM: optimizer with two coupled Sellar analyses and feedback exchange.



Case Study 1 — Sellar Validation Results



Case Study 2 — Airfoil Shape Optimization (SU2 + OMADS)

Problem setup from script 2d_airfoil.py:

$$\begin{aligned} \min_{\mathbf{x}} \quad & -C_L(\mathbf{x}) \\ \text{s.t.} \quad & |C_D(\mathbf{x})| - 0.006 \leq 0 \end{aligned}$$

Design variables (Bezier/B-spline control points)

- 10 shape parameters for upper/lower surface control points
- Typical bounds provided for each variable in OMADS setup
- Geometry regenerated each evaluation, then meshed via Gmsh

This is a blackbox, simulation-driven constrained optimization workflow for transonic airfoil design.

Evaluation loop

- SU2 CFD call: `mpiexec -n 5 SU2_CFD airfoil.cfg`
- Responses from `history.csv`: $C_L, C_D, L/D$
- OMADS poll settings: budget ≈ 200 , high precision



Case Study 3 — Supersonic Business Jet Problem

SBJ benchmark (DMDO/NHATC): A distributed multidisciplinary optimization problem for a supersonic business jet with strong couplings and nonlinear constraints.

 [Open in Google Colab](#)

- **Objective:** minimize aircraft weight while satisfying mission/performance requirements.
- **Disciplines:** aircraft performance, propulsion, aerodynamics, and structures.
- **Why it matters:** a compact but realistic benchmark for coordination under coupled constraints.

Context from Notebook

The formulation coordinates subsystem targets using NHATC in a distributed MDO setting.

Outline

- 1 Introduction
- 2 AI/ML in MDO
- 3 Case Studies
- 4 Tools & Frameworks**
- 5 Exercises
- 6 Future Directions



The MDO Software Ecosystem

MDO Frameworks

- **DMDO** — McGill (NHATC adopted by Siemens), open-source
- **OpenMDAO** — IRT Saint-Exupéry
- **HEEDSConnect** — Siemens DISW, Commercial

Optimization

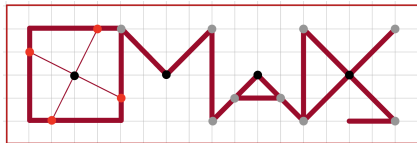
- OMADS, Pareto MO, SO
- NSGA-II
- Ax / BoTorch (Bayesian)

Surrogate & ML

- scikit-learn (GP, RBF)
- PyTorch / TensorFlow
- SMT — Surrogate Modeling Toolbox
- DeepXDE (PINNs)



Tools for today's exercises



Outline

- 1 Introduction
- 2 AI/ML in MDO
- 3 Case Studies
- 4 Tools & Frameworks
- 5 Exercises**
- 6 Future Directions



Workshop Exercises Overview

#	Exercise	Tools
1	LLM in MDO	scikit-learn
2	Sellar MDO	BoTorch
3	Coupled SBJ MDO	DMDO
5	PINN for heat transfer discipline	DeepXDE

Setup

All notebooks available at:

<https://github.com/Ahmed-Bayoumy/AIAA-WS2026-AIMDO>

```
pip install omads dmddo samplersLib pytorch deepxde
```



Outline

- 1 Introduction
- 2 AI/ML in MDO
- 3 Case Studies
- 4 Tools & Frameworks
- 5 Exercises
- 6 Future Directions**



Where Is the Field Heading?

Near-term (1–3 years)

- Foundation models for simulation (AeroGPT, ClimateNet)
- LLM-assisted problem formulation
- Uncertainty quantification as first-class citizen
- AI-driven collaborative MDO platforms

Long-term (5–10 years)

- Fully autonomous design agents
- Real-time MDO in digital twins
- Human-in-the-loop preference learning
- Quantum-enhanced optimization



Open Challenges

- ➊ **Scalability** — curse of dimensionality for $d > 100$ design variables
- ➋ **Extrapolation** — surrogates fail outside the training region
- ➌ **Certification** — regulators need explainable, verifiable AI-driven designs
- ➍ **Data scarcity** — high-fidelity runs are expensive; transfer learning is key
- ➎ **Multi-objective** — Pareto front approximation in black-box settings
- ➏ **Human factors** — bridging the gap between AI recommendations and designer intuition



Thank You!

Questions & Discussion



ahmed.bayoumy@siemens.com



github.com/Ahmed-Bayoumy/AIAA-WS2026-AIMDO



<https://ahmed-bayoumy.github.io/>

